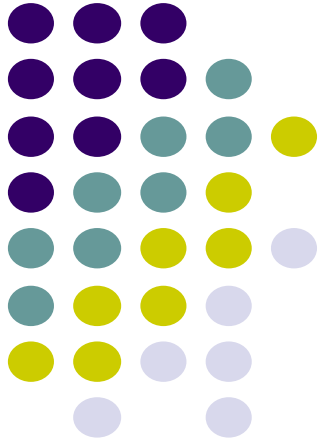
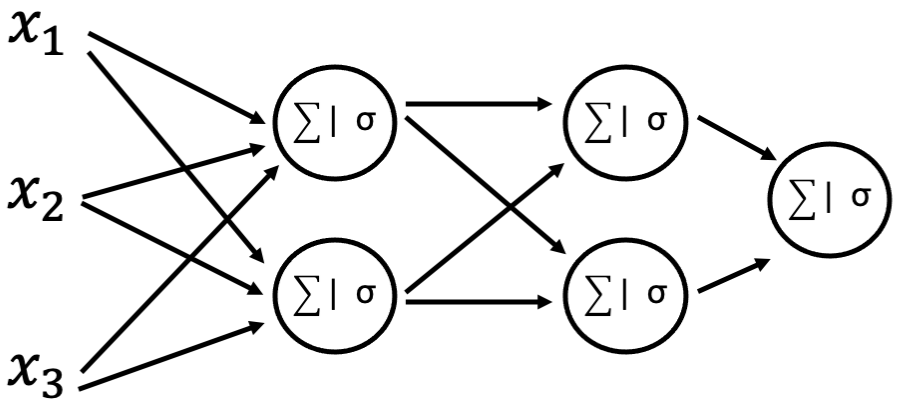
ErSE222: Machine learning in Geoscience

April. 5th, 2026

Tariq Alkhalifah and Omar Saad



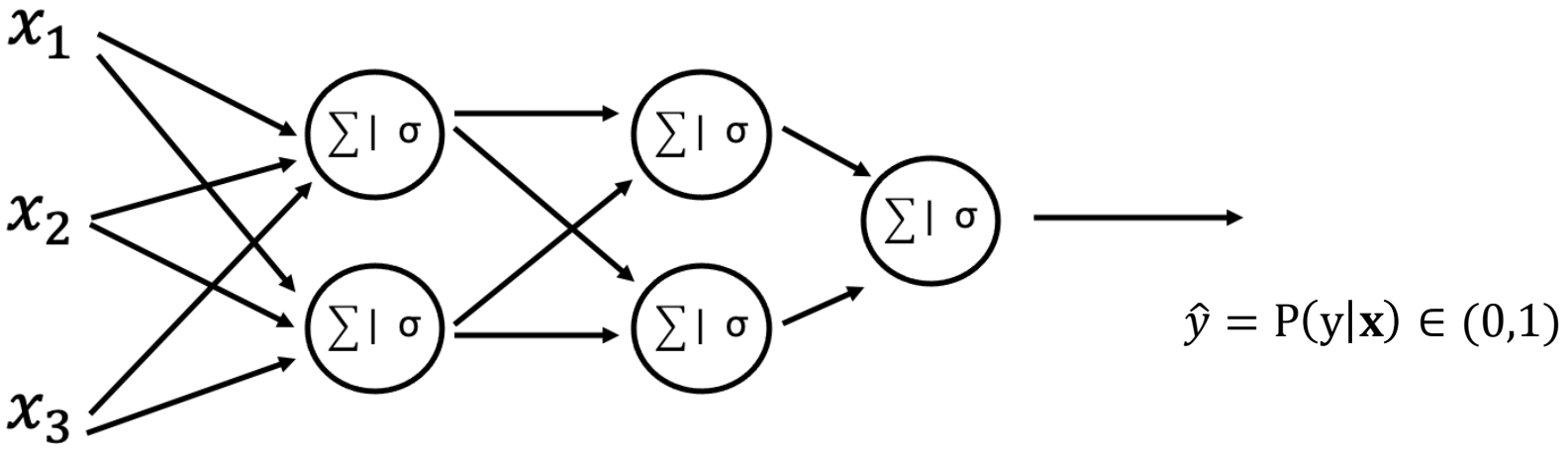
Deterministic vs probabilistic



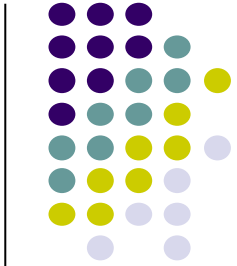
$\hat{y}$: Deterministic

$p(y)$ or $\hat{y}^{\{1\}}, \hat{y}^{\{2\}}, \hat{y}^{\{N\}}$: Probabilistic

Classification probability



Type of uncertainties in learning problems



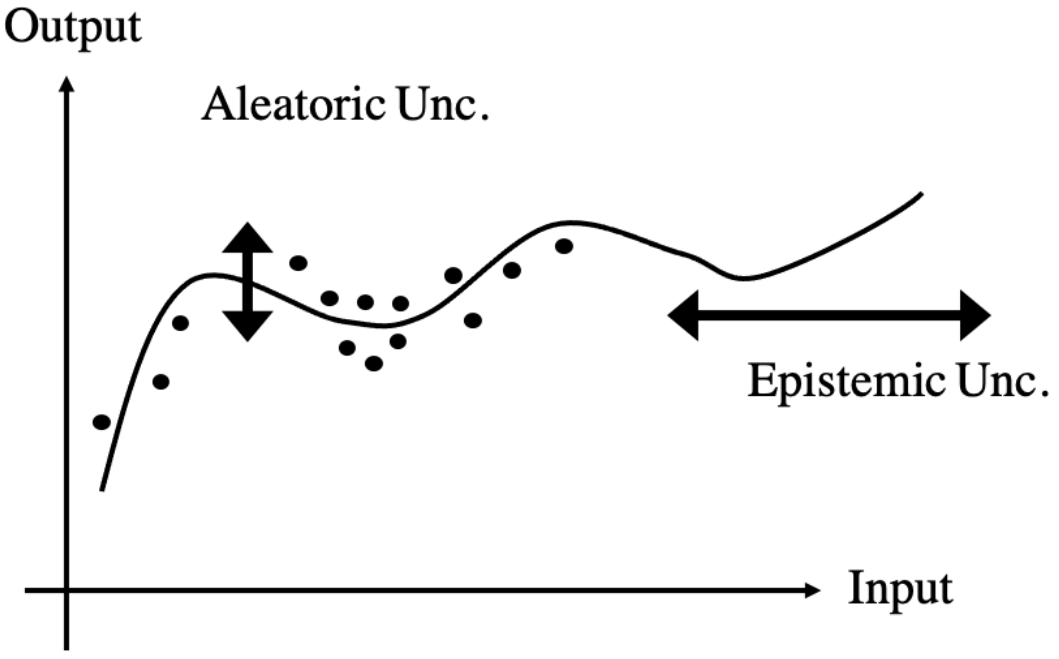
- Aleatoric uncertainty (or data uncertainty): input data may contain some intrinsic randomness

  - the function we try to approximate is **multimodal** (i.e., multiple possible outputs exist for a single input)
  - the recorded data is polluted by **noise**

- Epistemic uncertainty (or model uncertainty): lack of training data in certain regions of the input domain

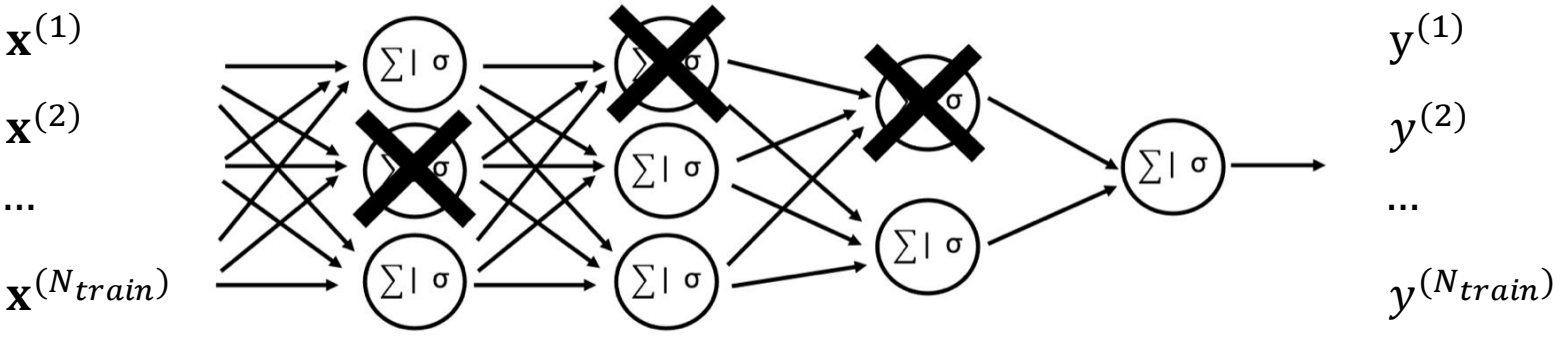
  - in regions populated with training data, the model has to learn an **interpolation** task
  - in regions not populated with training data, the model has to learn an **extrapolation** task

Type of uncertainties in learning problems



Quantifying uncertainties

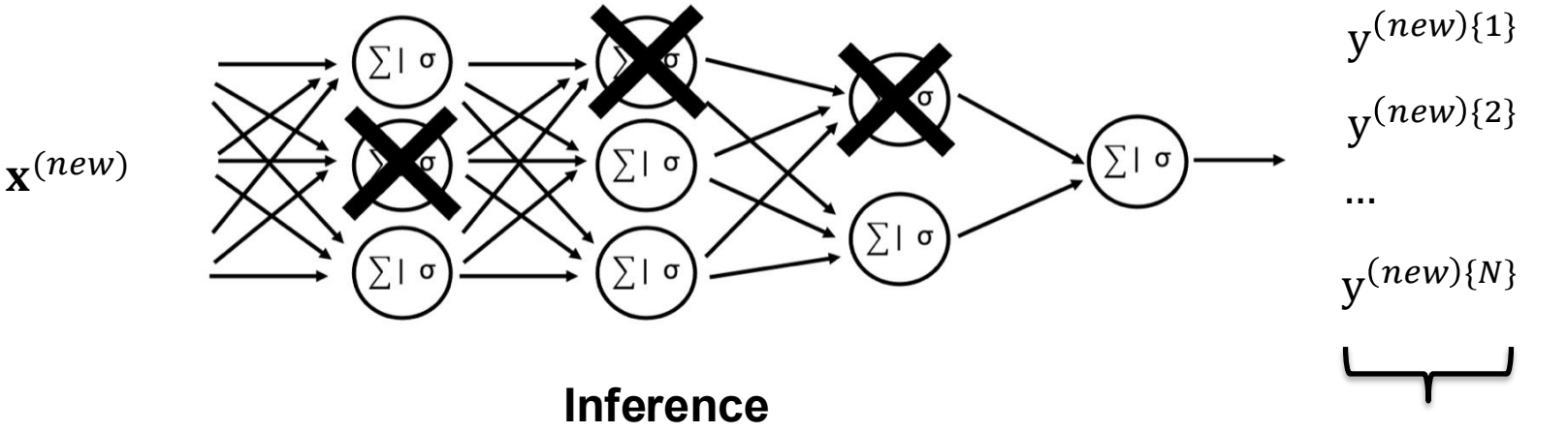
- Dropout at inference time



Training

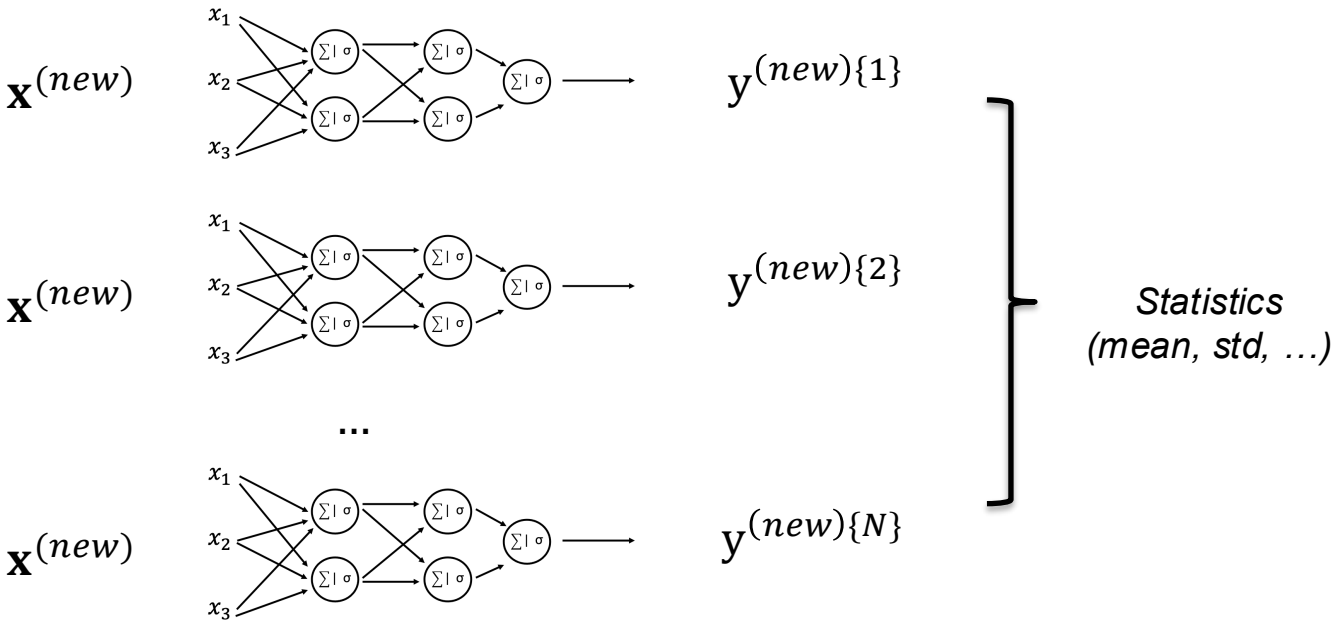
Quantifying uncertainties

- Dropout at inference time



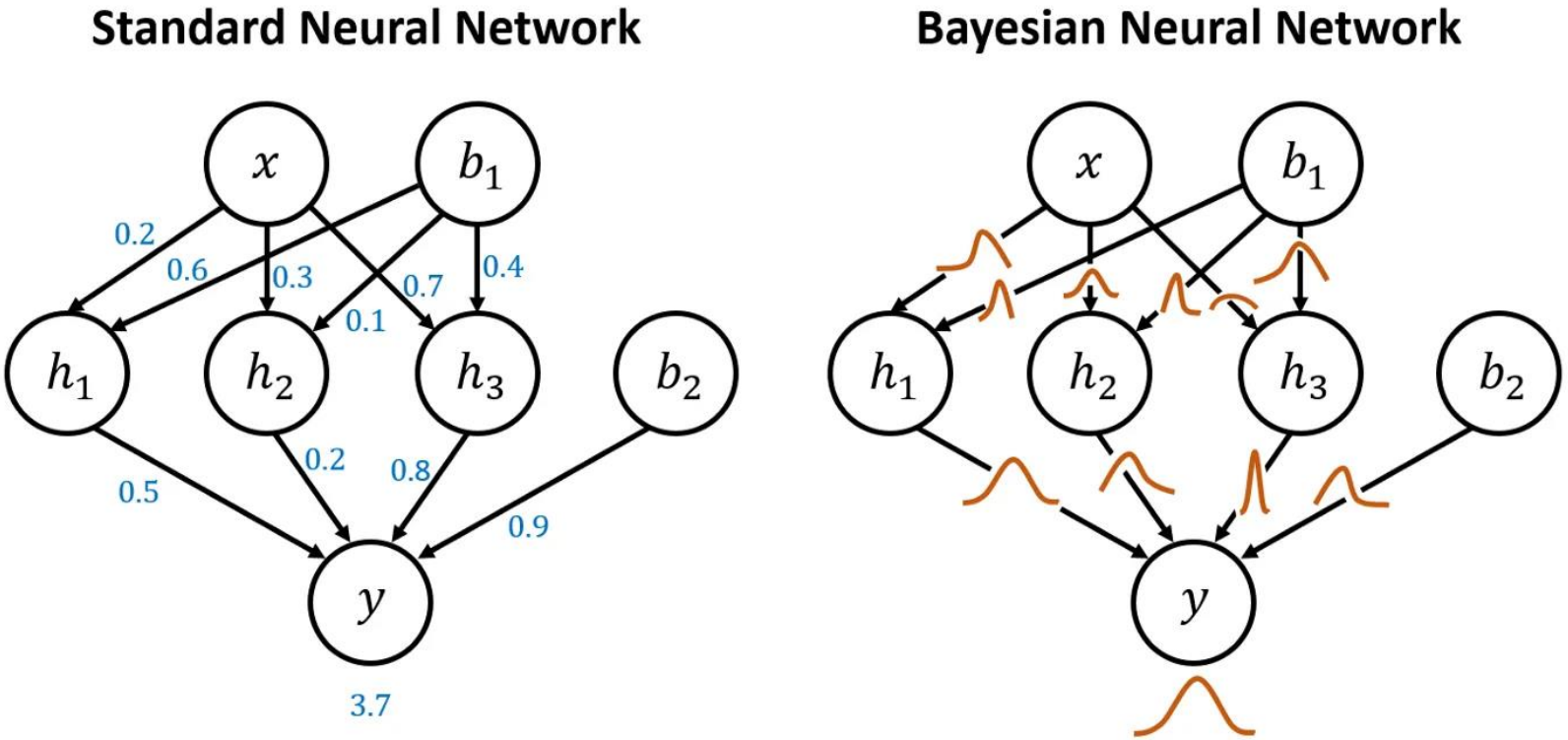
Quantifying uncertainties

- Ensembling: train multiple models in parallel



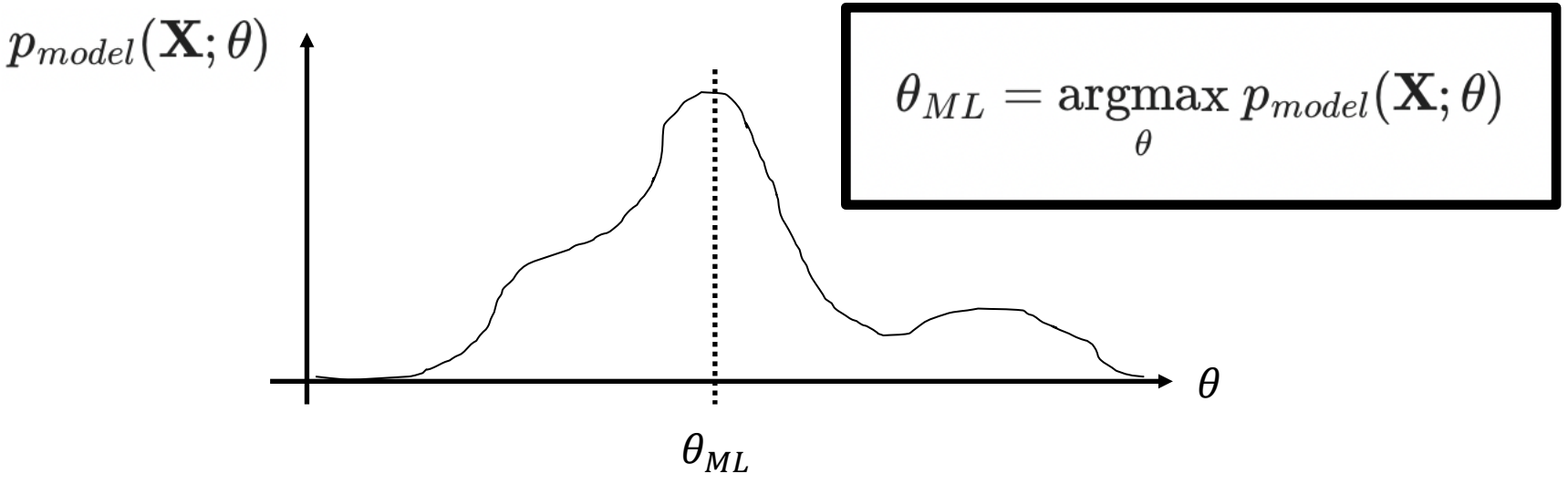
Bayesian Neural Network

A Bayesian Neural Network (BNN) is a type of neural network that incorporates principles from Bayesian statistics to model uncertainty in predictions. Unlike traditional neural networks that produce a single point estimate of the weights (and thus predictions), BNNs treat the weights as distributions. This approach allows for the quantification of uncertainty in the model's predictions.

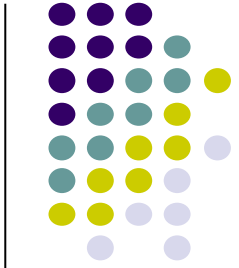


Maximum Likelihood Estimation (MLE)

Provides a unified view of the loss functions used in ML/DL



Maximum Likelihood Estimation (MLE)



Unknown true probability which we access indirectly via samples:

$$\mathbf{X} = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots, \mathbf{x}^{(N_s)}\} \sim p_{data}(\mathbf{X})$$

Known *parametric* probability (defined by us to mimic the true one): $p_{model}(\mathbf{X}; \theta)$

Ex: Gaussian distribution $p_{model}(\mathbf{x}; \{\boldsymbol{\mu}, \sigma\}) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{||\mathbf{x}-\boldsymbol{\mu}||_2^2}{2\sigma^2}}$ $\theta = \{\boldsymbol{\mu}, \sigma\}$

Maximum Likelihood Estimation (MLE)

$$\theta_{ML} = \underset{\theta}{\operatorname{argmax}}\, p_{model}(\mathbf{X};\theta)$$

From maximum likelihood to *minimum negative log-likelihood*:

$$\begin{aligned}
\theta_{ML} &= \underset{\theta}{\operatorname{argmax}} \prod_{i=1}^{N_s} p_{model}(\mathbf{x}^{(i)};\theta) \\
&= \underset{\theta}{\operatorname{argmax}} \sum_{i=1}^{N_s} log(p_{model}(\mathbf{x}^{(i)};\theta)) \\
&\approx \underset{\theta}{\operatorname{argmax}}\, E_{\mathbf{x}\sim p_{data}}[log(p_{model}(\mathbf{x};\theta))] \\
&= \underset{\theta}{\operatorname{argmin}}\, -E_{\mathbf{x}\sim p_{data}}[log(p_{model}(\mathbf{x};\theta))]
\end{aligned}$$

MLE for Learning problems

Consider conditional probability:

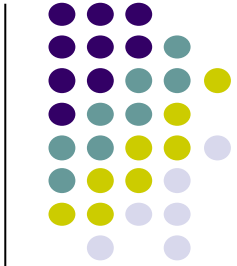
Target Input Parameters

$$\theta_{ML} = \underset{\theta}{\operatorname{argmax}}\, p_{model}(Y|\mathbf{X};\theta)$$

$$= \ldots$$

$$= \underset{\theta}{\operatorname{argmin}} \; - E_{\mathbf{x},y\sim p_{data}}[log(p_{model}(y|\mathbf{x};\theta))]$$

MLE for Classification



Binary classification:

$$p_{model}(y|\mathbf{x};\theta) = f_\theta(\mathbf{x})^y(1-f_\theta(\mathbf{x}))^{1-y}$$

$$logp_{model}(y|\mathbf{x};\theta) == y \cdot log\big(f_\theta(\mathbf{x})\big) + (1-y)\cdot log(1-f_\theta(\mathbf{x}))$$

$$\boldsymbol{\theta}_{ML} = \underset{\boldsymbol{\theta}}{\text{argmin}} \; -\sum_{i=1}^{N_s} log(p_{model}(y^{(i)}|\mathbf{x}^{(i)};\theta)$$

$$= -\sum_{i=1}^{N_s} y^{(i)} log\hat{y}^{(i)} + (1-y^{(i)})log(1-\hat{y}^{(i)})$$

→ Binary cross-entropy

Kullback–Leibler

KL divergence is a non-symmetric metric that measures the relative entropy or difference in information represented by two distributions.

$$D_{KL}(p(x)||q(x)) = \sum_{x \in X} p(x) \ln \frac{p(x)}{q(x)}$$

$$D_{KL}(p(x)||q(x)) = \int_{-\infty}^{\infty} p(x) \ln \frac{p(x)}{q(x)} dx$$

Bayesian Neural Network

Bayes' law provides a mathematical machinery to combine the observed training data with our prior modeling assumptions, such as the choice of model being a neural network, and the prior distribution over its weights:

posterior distribution

$$p(\omega|\mathcal{D}) = \frac{p(\mathcal{D}|\omega)p(\omega)}{p(\mathcal{D})},$$

evidence

$$p(\mathcal{D}) = \int p(\mathcal{D}|\omega)p(\omega)d\omega.$$

Bayesian Neural Network

The evidence is treated as a constant factor in previous Equation (it is intractable to compute the evidence for models with a large number of parameters). Thus, maximizing the posterior probability leads to

$$\omega_{MAP} = \underset{\omega}{\operatorname{argmax}}\{\log\ p(\mathcal{D}|\omega) + \log\ p(\omega)\}$$

$$= \underset{\omega}{\operatorname{argmin}}\{\mathcal{L}(\omega) - \log\ p(\omega)\},$$

Bayesian Neural Network (Inference)

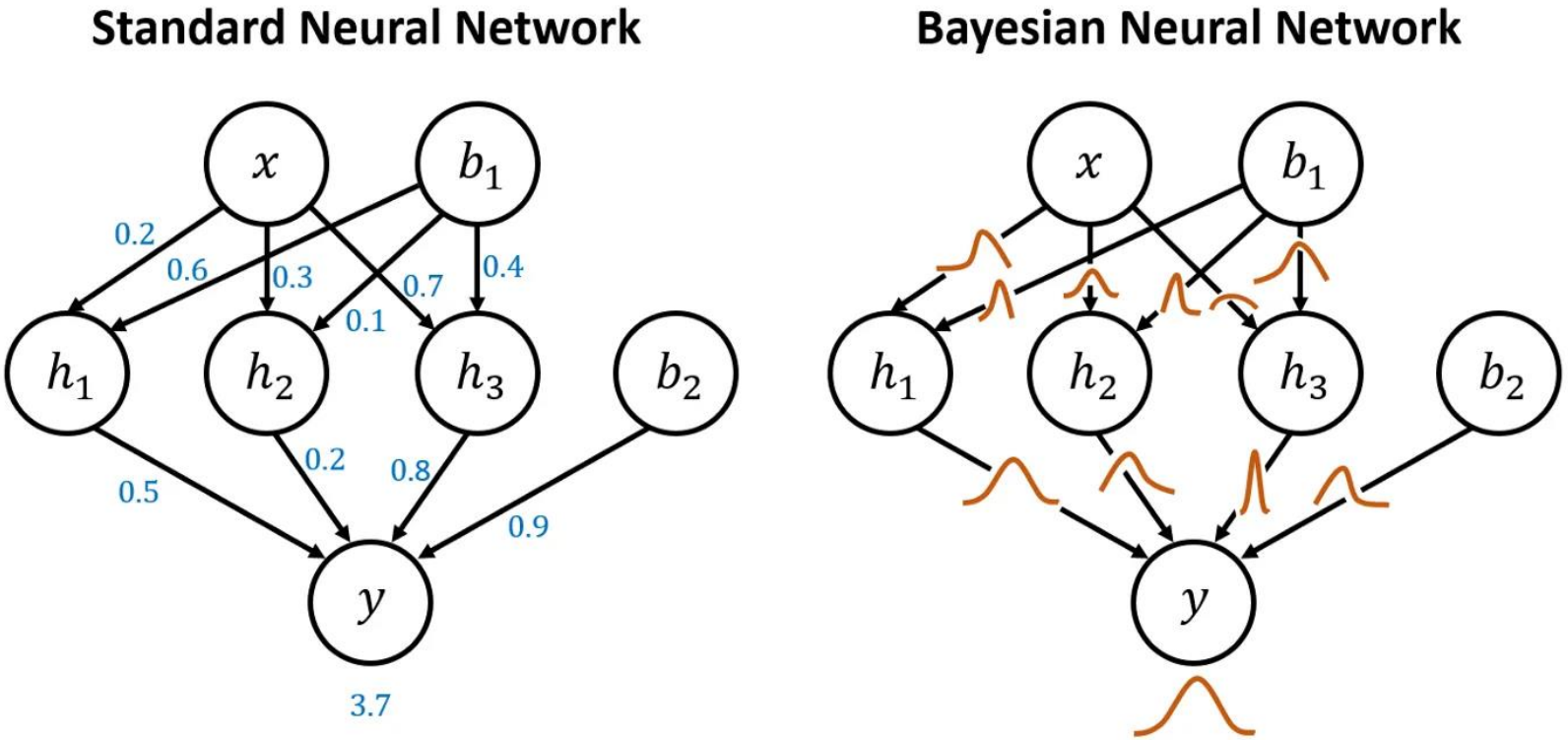
One can see that instead of taking a single set of weights that maximizes the probability of the posterior distribution, all possible weights are being considered and subsequently weighted by their probability. We can think of this as an ensemble of models weighted by the probability of each model. Indeed, this is equivalent to an ensemble of an infinite number of neural networks with the same architecture but with different weights.

$$p(\mathbf{y}^*|\mathbf{x}^*, \mathcal{D}) = \int_{\Omega} p(\mathbf{y}^*|\mathbf{x}^*, \omega) p(\omega|\mathcal{D}) d\omega$$

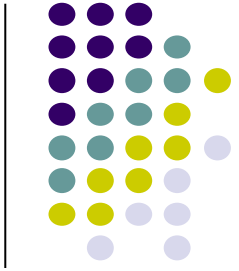
Neural Network

Bayesian Neural Network

A Bayesian Neural Network (BNN) is a type of neural network that incorporates principles from Bayesian statistics to model uncertainty in predictions. Unlike traditional neural networks that produce a single point estimate of the weights (and thus predictions), BNNs treat the weights as distributions. This approach allows for the quantification of uncertainty in the model's predictions.



Bayesian Neural Network



$\mathcal{D}_{tr} = (\mathbf{x}_i, y_i)_{i=1}^n$ Model: $F_{\boldsymbol{\theta}}()$ Loss: $\mathcal{L}()$ e.g. cross-entropy

Neural Network

Training:

Likelihood

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_{\text{tr}}} \log[p(y_i|\mathbf{x}_i, \boldsymbol{\theta})]$$

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_{\text{tr}}} \mathcal{L}(F_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i)$$

Prediction:

$$p(\hat{y}|\hat{\mathbf{x}}, \boldsymbol{\theta}^*)$$

$$\hat{y} = F_{\boldsymbol{\theta}^*}(\hat{\boldsymbol{x}})$$

Bayesian Neural Network

Training:

Regularization

$$\boldsymbol{\mu}^*, \Sigma^* = \arg\max_{\boldsymbol{\mu}, \Sigma} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_{\text{tr}}} \log[p(y_i|\mathbf{x}_i, \boldsymbol{\theta})] - \text{KL}[p(\boldsymbol{\theta}), p(\boldsymbol{\theta}_0)]$$

$$\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma) \qquad \boldsymbol{\theta}_0 \sim \mathcal{N}(\mathbf{0}, \mathrm{I})$$

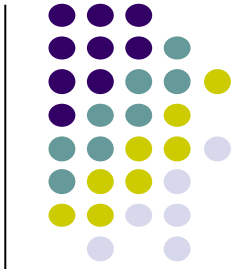
$$\boldsymbol{\mu}^*, \Sigma^* = \arg\min_{\boldsymbol{\mu}, \Sigma} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_{\text{tr}}} \mathcal{L}(F_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i) + \text{KL}[p(\boldsymbol{\theta}), p(\boldsymbol{\theta}_0)]$$

Prediction:

$$p(\hat{y}|\hat{\mathbf{x}}, \mathcal{D}_{\text{tr}}) = \int p(\hat{y}|\hat{\mathbf{x}}, \boldsymbol{\theta}^*) p(\boldsymbol{\theta}^*|\mathcal{D}_{\text{tr}}) d\boldsymbol{\theta}^*$$

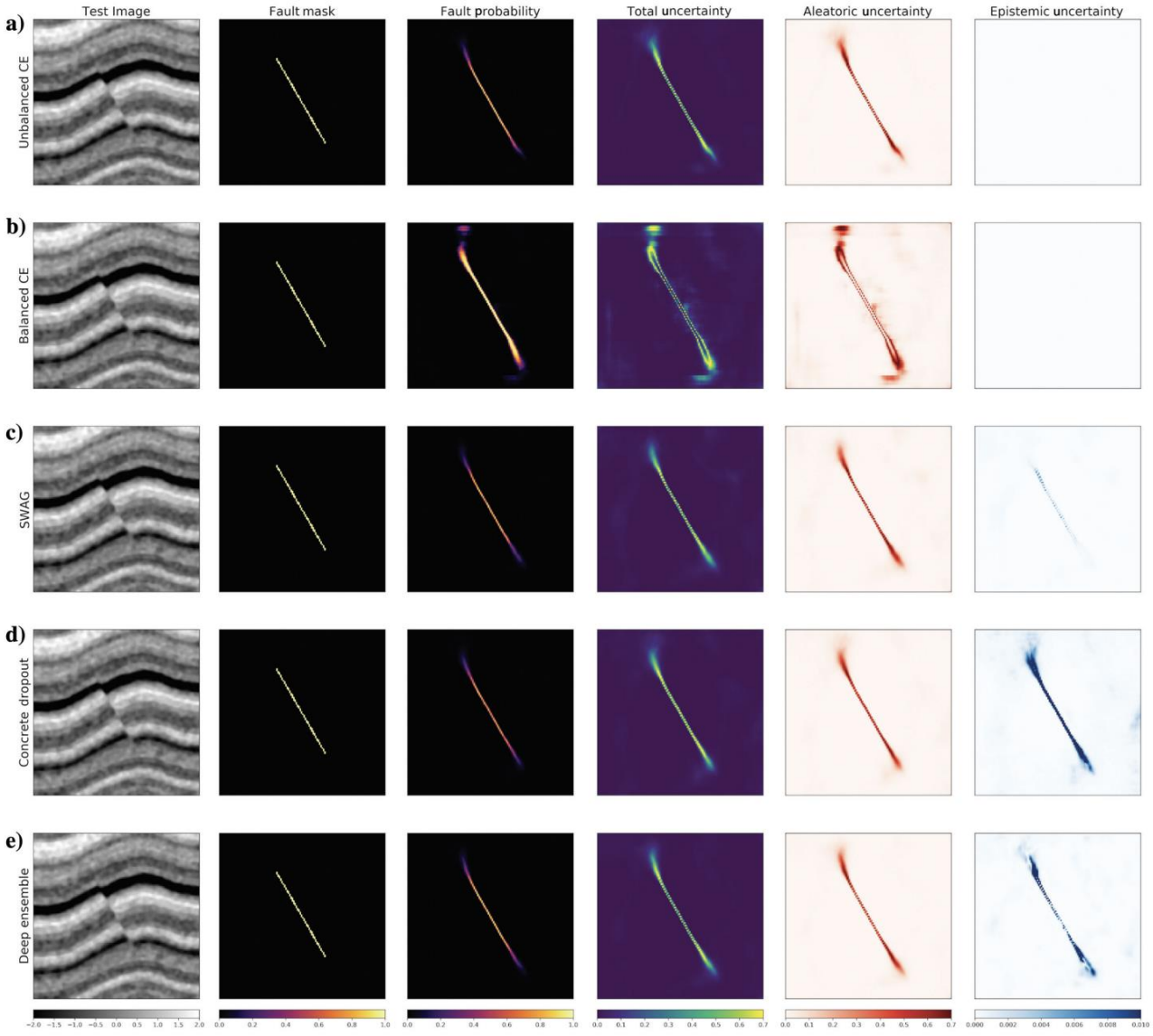
$$\boldsymbol{\theta}^* \sim \mathcal{N}(\boldsymbol{\mu}^*, \Sigma^*)$$

$$\hat{y} = \frac{1}{K} \sum_{k=1}^{K} F_{\boldsymbol{\theta}_k^*}(\hat{\boldsymbol{x}}) \qquad \boldsymbol{\theta}_k^* \sim \mathcal{N}(\boldsymbol{\mu}^*, \Sigma^*)$$



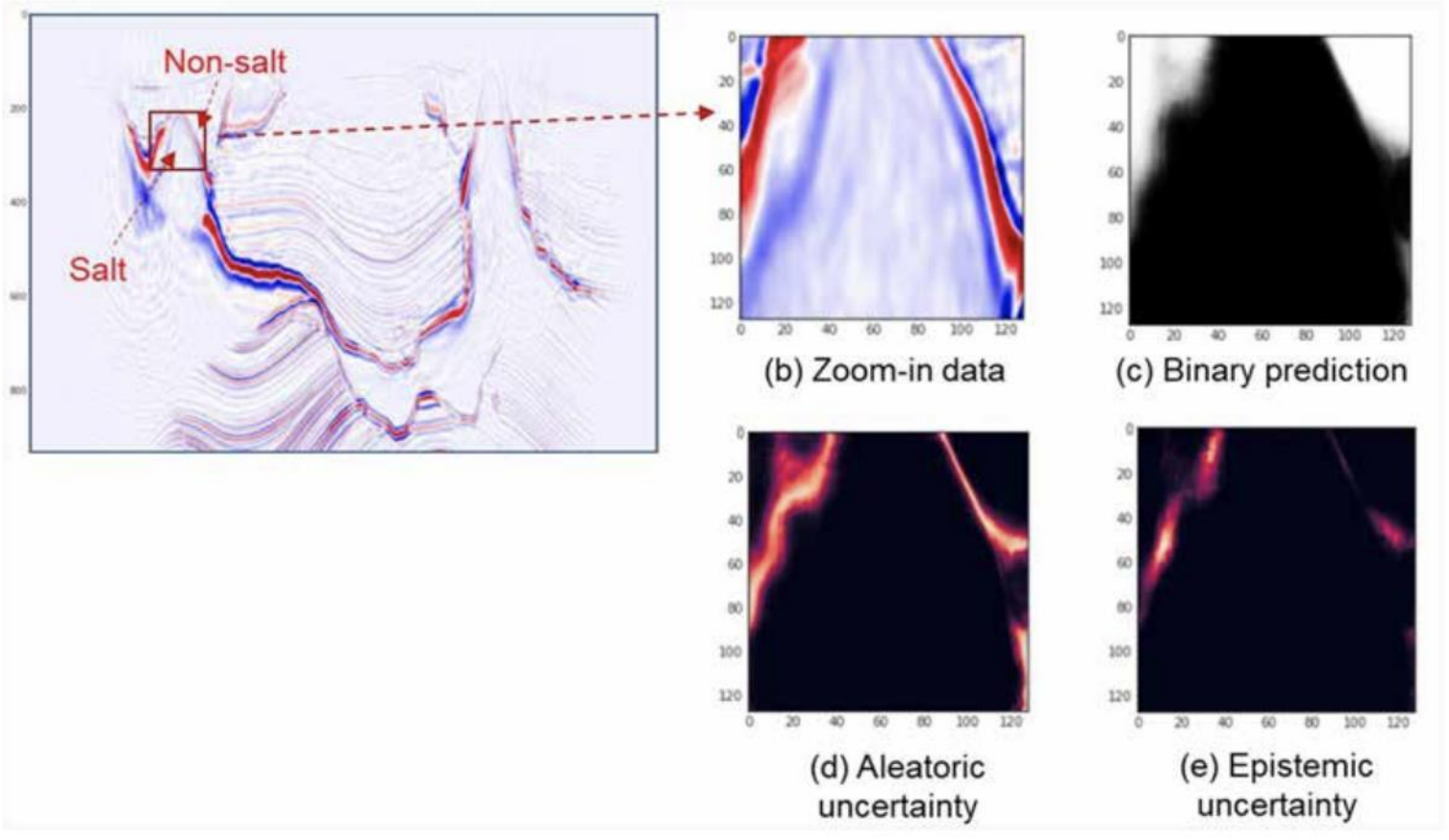
Application

Seismic Fault Detection



A comprehensive study of calibration and uncertainty quantification for Bayesian convolutional neural networks — An application to seismic data

Salt Bodies Segmentation



Uncertainty Analysis for Seismic Salt Interpretation by Convolutional Neural Networks